NELC Compliance Features

Developer Explanation Guide

Everything You Need to Know Before Writing Code

For: Development Team

From: Ghulam (Dev Lead)

Date: February 2025

What Are We Actually Building?

We are adding 6 NELC (National eLearning Center) compliance features to the PhysioGuidance platform. NELC is the Saudi government body that certifies eLearning platforms for the healthcare sector. Without these features, the platform cannot operate legally in Saudi Arabia.

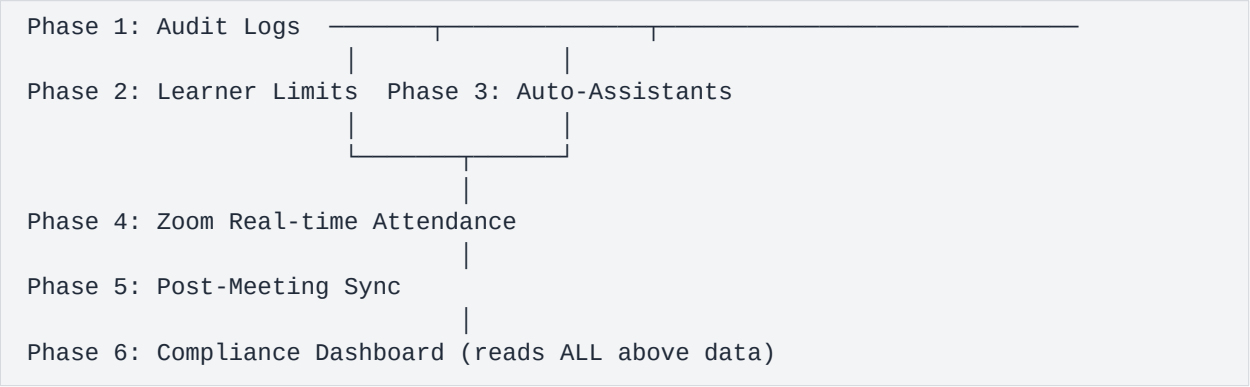
Think of it like this: just as a hospital needs medical accreditation to operate, an eLearning platform for healthcare training needs NELC certification. These 6 features are the technical requirements for that certification.

The 6 Features at a Glance

#	Feature	One-Line Explanation	Why NELC Needs It
1	Audit Logs	Immutable event log of everything that happens	Auditors need proof that rules were followed
2	Learner Limits	Cap enrollment per session (40 or 50 max)	Saudi regulation: classes cannot be too large
3	Auto-Assign Assistants	Auto-add teaching assistants when class is large	Large classes need extra supervision
4	Real-time Zoom Attendance	Track who joins/leaves Zoom by the minute	Prove students actually attended
5	Post-Meeting Sync	Reconcile Zoom reports with our records	Safety net: catch missed attendance
6	Compliance Dashboard	Visual overview of all compliance statuses	Quick audit readiness check

How Do They Connect?

Here is the dependency chain. You MUST build them in this order:



Why this order? Phase 1 (Audit Logs) is used by ALL other phases to record events. Phases 2 and 3 can be built in parallel since they are independent of each other. Phase 4 needs the attendance_logs table. Phase 5 builds on Phase 4's infrastructure. Phase 6 reads from everything.

Feature 1: NELC Audit Logs

What is it?

An immutable event log. Every important action in the system gets a log entry. Think of it like a security camera recording: it captures what happened, who did it, when, and from where. The key word is IMMUTABLE: once written, audit logs can NEVER be updated or deleted. This is a NELC requirement.

Why does NELC require it?

When NELC auditors review the platform, they need to verify that the platform followed all rules. For example: 'Did you actually enforce the 40-student limit on February 15th? Prove it.' The audit log shows: 'On Feb 15 at 14:32, enrollment was rejected for course X because it was at 40/40. Here is the exact error and metadata.'

How does it work technically?

A new PostgreSQL table called nelc_audit_logs. A global NestJS service that any module can inject. When sessions start/end, when attendance is marked, when enrollment is rejected, when assistants are assigned, a log entry is created automatically.

Critical rule: The NelcAuditService has only two methods: log() and query(). There is NO update() and NO delete(). This is intentional and must never change.

What events do we log?

Event	Triggered When	Example Metadata
SESSION_STARTED	Instructor clicks Start Session	{ sessionId, courseId, instructorId, method: 'zoom' }
ATTENDANCE_MARKED_MANUAL	Instructor marks student present/absent	{ studentId, sessionId, status: 'present' }
ATTENDANCE_ZOOM_JOIN	Student joins Zoom meeting	{ zoomEmail, joinTime, meetingId }
ENROLLMENT_LIMIT_ENFORCED	Enrollment rejected due to capacity	{ count: 40, max: 40, classification: 'long' }
ASSISTANT_ALLOCATED	System auto-assigns assistant	{ assistantId, sessionId, reason: 'auto' }
COMPLIANCE_CHECK_FAILED	Pre-start check finds issue	{ check: 'assistants', required: 2, actual: 1 }

What do you need to do?

- **Backend:** Create the table, service, module, and controller. Make the module global so any other module can inject it.

- **Integration:** Go into sessions.service.ts and enrollments.service.ts. Find the start(), end(), cancel(), markAttendance() methods. Add this.auditService.log(...) calls at the right points.
- **Frontend:** Add API methods in api.ts. No UI yet (that comes in Phase 6).

Feature 2: Learner Limits Enforcement

What is it?

NELC mandates maximum class sizes. Long courses (more than 5 days) can have at most 40 learners per session. Short courses (5 days or fewer) can have at most 50. The system must automatically enforce this by rejecting enrollment when a session is full.

The real-world analogy

Imagine a physical classroom with 40 chairs. When all chairs are taken, the next student is told 'Sorry, this class is full. Please join the next batch.' Our system does the exact same thing, but digitally.

Classification logic

Course Duration	Classification	Max Learners
>5 calendar days	Long	40
≤5 calendar days	Short	50

Important: Instructors/admins can manually set a lower limit (e.g., 25 for a small workshop), but they can NEVER exceed the NELC maximum. If an instructor sets maxEnrollment to 60 on a long course, the system caps it at 40.

Race condition warning

If 2 students click 'Enroll' at the exact same time when there is 1 seat left, both requests could pass the count check before either insert completes. This is a race condition. Solution: Use a PostgreSQL transaction with SELECT ... FOR UPDATE on the course row. This locks the row so the second request waits until the first completes.

Frontend changes

Show enrollment capacity bars on course cards (like '38/40 enrolled' with a progress bar). When near capacity, show an amber warning. When full, show a red 'Course Full' badge. On the create course form, auto-calculate the NELC limit based on dates and show it as a helpful info callout.

Feature 3: Auto-Assistant Allocation

What is it?

When a session has many enrolled students, NELC requires additional teaching assistants to be present. The system automatically assigns available instructors as assistants based on enrollment count.

The rules (pending client confirmation)

Enrolled Students	Required Assistants	What Happens
1-25	0	Just the main instructor
26-35	1	System auto-assigns 1 assistant from available pool
>35	2	System auto-assigns 2 assistants

How auto-assignment works

When a session is created or enrollment count crosses a threshold, the system checks if enough assistants are assigned. If not, it looks for available instructors (not already assigned to a session at that time) and assigns them. The assigned assistant gets a notification. Course managers can also manually assign/remove assistants.

New table: session_assistants tracks who is assigned where, whether it was auto or manual, and who made the assignment.

Feature 4: Real-time Zoom Attendance Tracking

> This is the most complex feature. Read carefully.

What is it?

When students attend a live Zoom session, we need to track exactly when each student joined and left, down to the minute. This data is used to calculate attendance duration and determine if students met the minimum attendance requirement (typically 70% of session time).

How Zoom webhooks work

Zoom can send HTTP POST requests to our server whenever something happens in a meeting. We subscribe to two events:

- **meeting.participant_joined —** Zoom tells us 'User X joined meeting Y at time Z'
- **meeting.participant_left —** Zoom tells us 'User X left meeting Y at time Z'

Our server receives these webhooks and processes them: match the Zoom participant to a student in our system (by email), record the event in attendance_logs, and update the attendance record.

The email matching challenge

The biggest risk: a student may register on our platform with ahmed@gmail.com but join Zoom with ahmed.work@company.com. We handle this with a matching chain:

- **Step 1:** Exact match with user.email
- **Step 2:** Exact match with user.zoomEmail (new field we are adding)
- **Step 3:** If no match, log the unmatched participant for manual resolution by the instructor

Duration calculation

A student might join, leave for lunch, rejoin, and then leave at the end. We track ALL join/leave pairs and calculate total time:

```
// Example: 2-hour session (09:00 - 11:00)
Join  09:05    Leave 10:15  (70 min)
Join  10:25    Leave 11:00  (35 min)
Total: 105 min out of 120 min = 87.5% attendance
```

Local development: Mock webhooks

We do NOT need a real Zoom account to develop this feature. We build a ZoomMockWebhookService that provides two endpoints:

- **POST /zoom/mock/simulate —** Simulate a single join or leave event for testing
- **POST /zoom/mock/full-session/:sessionId —** Simulate an entire session with realistic join/leave patterns for all enrolled students

These mock endpoints are ONLY available when NODE_ENV is not 'production'. They do not exist in production builds. This is enforced by a guard in the controller.

Feature 5: Post-Meeting Attendance Sync

What is it?

After a Zoom meeting ends, Zoom generates a detailed Participant Report that shows everyone who attended, when they joined/left, and how long they were in the meeting. This is more reliable than webhooks because webhooks can fail (network issues, server downtime). The sync service fetches this report and reconciles it with our attendance records.

Why do we need both webhooks AND sync?

Webhooks (Phase 4) give us real-time data. But they are not 100% reliable because of network issues, server restarts, or Zoom outages. The sync (Phase 5) is the safety net that catches anything webhooks missed. Together, they provide complete attendance data.

Reconciliation rules

- **Rule 1: Manual marks win.** If an instructor manually marked a student present, Zoom data does not override it.
- **Rule 2: Zoom supplements missing records.** If a student was in Zoom but has no attendance record, Zoom data creates one.
- **Rule 3: Duration is recalculated.** Total minutes are computed from all join/leave pairs.
- **Rule 4: Unmatched participants are logged.** If Zoom shows someone who is not in our system, it is flagged for instructor review.

How it runs

A cron job runs every 10 minutes. It finds sessions that ended more than 10 minutes ago (Zoom needs time to generate the report) and have not been synced yet. For each, it fetches the Zoom report and runs reconciliation. Instructors can also manually trigger a sync from the session detail page.

> Note: The Zoom Reports API requires a Zoom Business plan (not Pro). This is a client cost decision. In local development, we mock this API.

Feature 6: NELC Compliance Dashboard

What is it?

A dashboard that aggregates all compliance data from Phases 1-5 into a visual overview. Think of it as a health check for NELC readiness. Each session, course, and the platform as a whole gets a compliance score with green/amber/red indicators.

What does it check?

Check	GREEN	AMBER	RED
Enrollment limits	Under max	Within 5 of max	Over max (should never happen)
Assistant allocation	All assistants assigned	1 assistant short	2+ assistants short
Zoom settings	Waiting room + auth enabled	—	Missing security settings
Attendance compliance	>90% students meet 70% threshold	70-90% meet threshold	<70% meet threshold
Audit trail completeness	All events logged	Minor gaps	Critical events missing

Where does it show up?

- **Course Manager Dashboard:** NELCComplianceWidget (already exists as mock, wire to real data)
- **Instructor Dashboard:** Pre-start compliance check modal before starting a session
- **Admin Dashboard:** Full compliance detail page with drill-down into courses and sessions
- **Session cards everywhere:** Small green/amber/red dot indicating compliance status

Cross-Dashboard Impact

These features are being built for the Instructor Dashboard first, but the backend services will be consumed by other dashboards too. Here is the full picture:

Feature	Instructor	Course Manager	Admin	Learner
Audit Logs	Auto-logged on actions	Views compliance reports	Full audit log UI + exports	—
Learner Limits	Sees capacity bar	Sets/overrides limits	Policy configuration	'Course Full' message
Auto-Assistants	Sees assignments, gets assigned	Manages allocation	Configures rules	—
Zoom Attendance	Real-time tracking UI	Attendance reports	Compliance KPI cards	Sees own attendance %
Compliance Dashboard	Pre-start check	NELC widget (exists)	Full compliance matrix	—

Design the API endpoints to be role-agnostic. Same endpoint, different data based on the user's role. For example, GET /nelc-compliance/summary returns all courses for Admin, but only the instructor's courses for Instructor role.

Local Testing Strategy

You can develop and test everything locally without paying for any third-party service. Here is how:

Phase	External Service Needed?	Local Strategy
1. Audit Logs	None	100% local. Just PostgreSQL.
2. Learner Limits	None	100% local. Just business logic + DB.
3. Auto-Assistants	None	100% local. Just business logic + DB.
4. Zoom Attendance	Zoom Pro (for real testing)	Use ZoomMockWebhookService. POST /zoom/mock/simulate to test. POST /zoom/mock/full-session/:id for end-to-end.
5. Zoom Sync	Zoom Business (for real testing)	Use ZoomMockReportService. Returns realistic participant data. Test the reconciliation logic fully.
6. Compliance Dashboard	None	100% local. Reads from data created in Phases 1-5.

How to Test Locally (Step by Step)

1. Set ZOOM_MOCK=true in your .env.development file.

2. Run the backend: npm run start:dev

3. Create a course with start/end dates (this triggers learner limit calculation).

4. Enroll students until you hit the limit. Verify rejection message.

5. Create a session. Check if assistants are auto-assigned based on enrollment count.

6. Simulate a Zoom session:

```
# Simulate student joining
curl -X POST http://localhost:3000/zoom/mock/simulate \
  -H 'Content-Type: application/json' \
  -d '{"sessionId": "<uuid>", "studentEmail": "test@example.com", "event":
"join"}'

# Simulate full session with all enrolled students
curl -X POST http://localhost:3000/zoom/mock/full-session/<sessionId>

# Trigger post-meeting sync
curl -X POST http://localhost:3000/sessions/<sessionId>/sync-zoom

# Check compliance
curl http://localhost:3000/nelc-compliance/session/<sessionId>
```

7. Check audit logs: GET /nelc-audit?eventType=ATTENDANCE_ZOOM_JOIN

8. Check compliance dashboard: GET /nelc-compliance/summary

Coding Standards for These Features

1. Interface-first for external services

Every external service (Zoom, SMS, WhatsApp) must have an interface. Business logic injects the interface, never the concrete class. This is how we swap between mock and real:

```
// CORRECT:
constructor(@Inject('IZoomService') private zoom: IZoomService) {}

// WRONG:
constructor(private zoom: ZoomService) {}
```

2. Always audit log

If you are writing a service method that creates, updates, or deletes something important, add an audit log. When in doubt, log it. It is better to have too many logs than to miss a critical event during a NELC audit.

3. All timestamps in UTC

Store everything in UTC in the database. The frontend converts to Saudi Arabia time (UTC+3) for display. Zoom sends timestamps in UTC, our cron jobs run in UTC. Never store local time.

4. Bilingual error messages

All user-facing error messages need both Arabic and English versions. Use the existing i18n system. Backend sends error codes, frontend looks up the localized message.

5. Export support on every data endpoint

Every GET endpoint that returns a list should support ?format=csv and ?format=pdf query parameters. This is a NELC requirement for evidence export. Use the existing export utilities.

6. RBAC on every endpoint

Use the @Roles() decorator on every controller method. No endpoint should be accessible without role validation. The existing RolesGuard handles this.

Questions?

If anything is unclear, ask Ghulam before writing code. It is much faster to clarify upfront than to refactor later. Especially for Phases 4 and 5 (Zoom integration), make sure you understand the mock service pattern before starting.